

All 30 commands

Start Plan Build Check + fix Explain Memory Experts

/sc:recommend

👁 LOOK ONLY

Not sure where to start? Describe your goal in plain words and it lists which commands to run, in order.

/sc:help

👁 LOOK ONLY

Shows a quick table of commands and flags. Handy reminder, though it only lists 24 of the 30.

/sc

👁 LOOK ONLY

Type /sc on its own to see a short menu of everything SuperClaude can do.

/sc:pm

✍ CHANGES FILES

Your project manager. Describe a goal in plain English; it plans the work and calls in the right experts.

/sc:agent

✍ CHANGES FILES

Runs a whole task for you: checks git, investigates, builds, then reviews its own work before finishing.

/sc:task

✍ CHANGES FILES

Does one clearly described job from start to finish, then stops and waits for you.

/sc:spawn

📄 PLANS ONLY

Breaks a very big job into smaller pieces and decides what can run side by side. Plans only.

/sc:select-tool

👁 LOOK ONLY

Picks the right tool for an edit: Serena for smart renames, Morphllm for quick bulk replacements.

/sc:brainstorm

📄 PLANS ONLY

Asks you friendly questions until a vague idea becomes a clear list of what to build.

/sc:estimate

👁 LOOK ONLY

Tells you roughly how long or how hard something will be, broken down piece by piece.

/sc:workflow

📄 PLANS ONLY

Turns your requirements into a step-by-step build plan in the right order. Writes no code.

/sc:design

📄 PLANS ONLY

Plans the structure first: overall system, API, a screen component or a database. No code yet.

/sc:implement

✍ CHANGES FILES

The main "build it" command. Writes the actual code for a feature, with tests if you ask.

/sc:build

✍ CHANGES FILES

Compiles and packages your project with the setup you already have, and reports any errors.

/sc:improve

✍ CHANGES FILES

Makes working code cleaner, faster or easier to maintain. Asks before any bigger change.

/sc:cleanup

✍ CHANGES FILES

Removes unused code, imports and files to keep the project tidy. Asks when it is unsure.

/sc:git

✍ CHANGES FILES

Handles git for you and writes clear commit messages. Asks before anything risky.

/sc:analyze

👁 LOOK ONLY

Reads your code and lists problems by severity: quality, security, speed or structure. Never edits.

/sc:test

👁 LOOK ONLY

Runs the tests you already have and shows how much of your code they actually check.

/sc:troubleshoot

👁 LOOK ONLY

Finds out why something is broken and suggests fixes. Changes nothing unless you add --fix.

/sc:reflect

👁 LOOK ONLY

A sanity check: are we still solving the right problem, and is the task really finished?

/sc:explain

👁 LOOK ONLY

Explains code or an idea in plain language, at beginner, intermediate or advanced level.

/sc:document

✍ CHANGES FILES

Writes documentation for one part of your project: code comments, an API reference or a guide.

/sc:index

✍ CHANGES FILES

Creates documentation for the whole project, such as a README or a full docs overview.

/sc:index-repo

✍ CHANGES FILES

Makes a short map of your project so later sessions read far less and cost fewer tokens.

/sc:load

📄 MEMORY

Start of a session: reopens your project and restores what was saved last time.

/sc:save

📄 MEMORY

End of a session: saves progress and lessons, so next time picks up exactly where you left off.

/sc:research

📄 PLANS ONLY

Searches the web and writes a report with sources, so your decisions rest on evidence.

/sc:business-panel

📄 PLANS ONLY

Nine famous business thinkers (simulated) discuss or debate your plan and give advice.

/sc:spec-panel

📄 PLANS ONLY

Simulated software experts score your specification and point out what is missing or unclear.

👤 All 19 specialists (agents)

Type the name to call one. Most also switch on by themselves.

@agent-backend-architect

Designs the behind-the-scenes part: servers, APIs and databases that stay secure and never lose data.

Use for APIs, databases, login

@agent-frontend-architect

Builds what people see and click: fast, works on every screen size, usable by everyone.

Use for Screens, accessibility

@agent-system-architect

Plans the big picture so your system keeps working when it grows ten times bigger.

Use for Structure, scaling

@agent-devops-architect

Automates testing, releasing and monitoring, so going live is routine instead of risky.

Use for Pipelines, servers

@agent-python-expert

Writes clean, tested, secure Python that is ready for real use from the first draft.

Use for Python projects

@agent-security-engineer

Thinks like a hacker to find weak spots before someone with bad intentions does.

Use for Security reviews

@agent-quality-engineer

Looks past the normal case for the unusual situations that break things, and plans the tests.

Use for Test plans

@agent-performance-engineer

Measures what is slow before changing anything, then proves the speed-up with numbers.

Use for Speed, memory

@agent-refactoring-expert

Tidies messy code in small, safe steps without changing what it does.

Use for Clean-ups

@agent-self-review

Checks finished work like a reviewer: were tests run, edge cases covered, requirements met?

Use for Before "done"

@agent-root-cause-analyst

A detective for hard bugs: follows the evidence and tests theories instead of guessing.

Use for Tricky bugs

@agent-requirements-analyst

Keeps asking "why?" until a vague wish becomes a clear, testable specification.

Use for Specs, user stories

@agent-deep-research-agent

Researches like a journalist: searches widely, follows leads and cites every source.

Use for /sc:research

@agent-technical-writer

Writes clear documentation for the people who will actually read it.

Use for Docs, guides

@agent-learning-guide

Teaches new topics step by step, connecting them to what you already know.

Use for Learning

@agent-socratic-mentor

Coaches you with questions, so you work out the answer yourself and remember it.

Use for Skill building

@agent-pm-agent

Keeps a work session on track: remembers context, plans, saves checkpoints and learns from mistakes.

Use for Long sessions

@agent-repo-index

Shrinks your whole project into a short map, so Claude reads less and costs less.

Use for Saving tokens

@agent-business-panel-experts

Provides the nine simulated business thinkers used by /sc:business-panel.

Use for Strategy reviews

7 modes

Working styles. They switch on by themselves, or force one with the flag.

Brainstorming

--brainstorm

Asks questions instead of guessing while your request is still vague.

Introspection

--introspect

Explains its thinking step by step so you can see why it chose something.

Deep Research

/sc:research

Plans an investigation, searches many sources and tells you how sure it is.

Task Management

--task-manage

Splits big jobs into phases, tracks progress and saves checkpoints.

Orchestration

--orchestrate

Picks the best tool for each step and runs independent steps at once.

Token Efficiency

--uc

Switches to short, compressed answers when the chat is nearly full.

Business Panel

/sc:business-panel

Simulated business experts discuss, debate or question your strategy.

10 helper tools (MCP)

Optional add-ons. Install: `superclaude mcp --servers NAME`

Context7

Looks up the official, up-to-date documentation for the libraries you use.

--c7

 **NODE 18+**

Magic

Generates modern, ready-made interface parts like buttons, forms and menus.

--magic

 **API KEY**

Chrome DevTools

Measures how fast a web page loads and spots errors in the browser console.

--chrome

 **NODE 18+**

Morphllm

Makes the same change across many files, quickly and cheaply.

--morph

 **API KEY**

Airis Agent

Double-checks decisions and adds research tools, via the AIRIS gateway.

via gateway

 **DOCKER**

Sequential

Helps Claude reason through complex problems one careful step at a time.

--seq

 **NODE 18+**

Playwright

Opens a real browser to click through your app and test it like a user would.

--play

 **NODE 18+**

Serena

Understands code by meaning for safe renames, and remembers your sessions.

--serena

 **UV**

Tavily

Searches the web for current information beyond Claude's training.

--tavily

 **API KEY**

MindBase

Long-term memory you can search by meaning, via the AIRIS gateway.

via gateway

 **DOCKER**

Flags

Small switches you add to the end of any command.

Behaviour modes

--brainstorm Ask questions first, assume nothing

--introspect Show the reasoning behind each decision

--task-manage Break into phases and track progress

--orchestrate Choose tools carefully, run independent steps in parallel

--token-efficient Compressed replies, 30–50% fewer tokens

Tool switches: on

--c7 / **--context7** Official documentation lookup

--seq / **--sequential** Structured multi-step reasoning

--magic UI component generation

--morph / **--morphllm** Fast pattern edits across files

--serena Meaning-aware code tools and memory

--play / **--playwright** Real-browser testing

Tool switches: more

--chrome / **--devtools** Live page profiling

--tavily Web search

--frontend-verify Playwright + DevTools + Serena together

--all-mcp Every tool at once

--no-mcp No external tools; built-ins and web search only

Thinking depth

--think Standard structured analysis, about 4K tokens. Adds Sequential

--think-hard Deep analysis, about 10K tokens. Adds Sequential and Context7

--ultrathink Maximum depth, about 32K tokens. Adds every tool

Execution control

--delegate Split work across parallel sub-agents

--concurrency Cap how many things run at once

--loop Repeat improve-and-check cycles

--iterations Set the number of cycles

--validate Assess risk before acting

--safe-mode Most cautious execution; also turns on **--uc**

Output and focus

--uc / **--ultracompressed** Symbol-based, very short replies

--scope How wide to look

--focus One area of concern

P1 🌐 WEB ⚡ POWER

Dark mode and theming

Add a light and dark theme that switches cleanly and keeps text readable in both.

- 1 `/sc:design themes --type component --format spec "light + dark colour tokens, contrast, user preference" # theme plan`
- 2 `/sc:implement "theme tokens + light/dark switcher" --type component --magic --c7 --with-tests # build it`
- 3 `/sc:improve src --type style --morph "replace hard-coded colours with theme tokens" # apply everywhere`
- 4 `/sc:test --type e2e --frontend-verify # check both themes`

P2 🌐 WEB ⚡ POWER

Accessibility compliance sweep

Make sure people who use screen readers or only a keyboard can use your site.

- 1 `/sc:analyze src --focus quality --depth deep "WCAG 2.1 AA: contrast, labels, focus, keyboard" # issues`
- 2 `/sc:improve src --type quality --focus accessibility --interactive # fixes`
- 3 `/sc:test --type e2e --play # browser checks`

P3 🌐 WEB ⚡ POWER

Redesign an existing site to modern standard

The site works but looks dated. Audit it, define a fresh style, restyle it and test.

- 1 `/sc:analyze src --focus quality --depth deep "visual consistency, accessibility, responsiveness" # audit`
- 2 `/sc:design design-system --type component --format spec # new system`
- 3 `/sc:improve src --type style --magic --loop --iterations 3 # restyle`
- 4 `/sc:test --type e2e --frontend-verify # verify`

P4 📦 MAINTENANCE ⚡ POWER

Dependency upgrade sweep

Packages are outdated or have known security holes. Update them safely.

- 1 `/sc:analyze . --focus security "outdated and vulnerable dependencies" # inventory`
- 2 `/sc:research "breaking changes in <packages>" --depth quick # changeLogs`
- 3 `/sc:improve src --type maintainability --morph --interactive # update usages`
- 4 `/sc:test --type all # verify`

P5 🌐 WEB ⚡ POWER

Extract and enforce a design system

Colours, spacing and buttons differ from page to page. Gather them into one system and apply it everywhere.

- 1 `/sc:analyze src --focus quality "inventory colours, fonts, spacing, component variants" # inventory`
- 2 `/sc:design tokens --type component --format code # tokens`
- 3 `/sc:improve src --type style --morph "replace hard-coded values with tokens" # enforce`
- 4 `/sc:test --type e2e --play # check`

P6 📖 ONBOARDING ⚡ POWER

Take over an unfamiliar project in a day

You inherited code with no handover. Understand it within a day.

- 1 `/sc:index-repo # map`
- 2 `/sc:analyze . --focus architecture --depth quick # overview`
- 3 `/sc:explain src --level basic # tour`
- 4 `/sc:test --type all # what works`
- 5 `/sc:save --type learnings # remember`

P7 📱 MOBILE ⚡ POWER

Mobile feature end to end

Add one new feature to a phone app, including the server part it needs.

- 1 `/sc:design feature-api --type api --format spec # contract`
- 2 `/sc:implement "<feature> endpoint" --type api --with-tests # API`
- 3 `/sc:implement "<feature> screen" --type component --framework react --with-tests # screen`
- 4 `/sc:test --type all --coverage # verify`

P8 📅 PLANNING ⚡ POWER

Vague request to build-ready ticket

Someone asks for something fuzzy. Turn it into a clear, sized, step-by-step ticket before anyone writes code.

- 1 `/sc:brainstorm "<the request>" -depth shallow # clarify what is wanted`
- 2 `/sc:spec-panel @claudedocs/requirements.md --mode critique --focus requirements # find the gaps`
- 3 `/sc:estimate "<the request>" --type effort --breakdown # how big it is`
- 4 `/sc:workflow "<the request>" --strategy agile --depth shallow # steps to build it`

P9 MOBILE POWER

Mobile app performance + accessibility audit

The app feels slow or is hard to use for some people. Find the causes, then fix them.

- 1 `/sc:analyze src --focus performance --depth deep "startup, re-renders, list virtualisation, images" # perf`
- 2 `/sc:analyze src --focus quality "touch targets, contrast, screen-reader labels" # a11y`
- 3 `/sc:improve src --type performance --interactive # fixes`
- 4 `/sc:test --type all # verify`

P10 RESEARCH POWER

Technology decision with evidence

Choosing a framework, database or vendor. Decide with evidence, not hunches.

- 1 `/sc:recommend "<need>" --alternatives # options`
- 2 `/sc:research "<A> vs <B> for <use case> 2026" --depth deep # evidence`
- 3 `/sc:business-panel @claudedocs/research_<topic>.md --mode debate # stress test`
- 4 `/sc:design <system> --type architecture --format spec # decision`

P11 DEBUG POWER

Production incident: diagnose, fix, postmortem

Something is broken for real users right now. Fix it safely, then write down what happened.

- 1 `/sc:git checkout -b hotfix/<issue> # branch`
- 2 `/sc:troubleshoot "<symptom since time>" --type deployment --trace --think-hard # diagnose`
- 3 `/sc:troubleshoot "<symptom>" --fix --safe-mode # careful fix`
- 4 `/sc:test --type all # verify`
- 5 `/sc:document claudedocs/ --type external --style detailed "postmortem" # postmortem`

P12 AI / ML POWER

Build a model / prompt evaluation harness

You change a model or prompt but can't tell if it got better. Build a way to measure it.

- 1 `@agent-quality-engineer "design an eval strategy for <model or prompt>" # strategy`
- 2 `/sc:implement "eval harness with golden set and metrics" --type service --with-tests # harness`
- 3 `/sc:test --type all # baseline scores`

P13 DEBUG POWER

Crash or regression: find the cause, fix it

Something crashed, or worked last week and now doesn't. Find the cause, even the exact commit, then fix it.

- 1 `/sc:troubleshoot "<paste error + stack trace>" --type bug --trace # likely cause`
- 2 `/sc:git log # what changed recently`
- 3 `/sc:git bisect start # pinpoint the breaking commit`
- 4 `/sc:troubleshoot "<error>" --type bug --fix # fix`
- 5 `/sc:test --type all # verify`

P14 DOCS POWER

Complete documentation set

Documentation is missing or out of date. Create a complete, current set.

- 1 `/sc:index . --type readme --format md # README`
- 2 `/sc:document src/api --type api --style detailed # API ref`
- 3 `/sc:document src --type guide --style detailed # guides`
- 4 `/sc:index . --type docs --format md # docs index`

P15 AI / ML POWER

Add an LLM / RAG feature

Let users chat with, or search, your own documents and data using an AI model.

- 1 `/sc:research "RAG best practice 2026: chunking, retrieval, evals" --depth deep # approach`
- 2 `/sc:design rag --type architecture --format spec # design`
- 3 `/sc:implement "retrieval pipeline + answer endpoint" --type service --with-tests --c7 # build`
- 4 `/sc:analyze src/rag --focus security "prompt injection, data leakage" # safety`

P16 PERFORMANCE POWER

Core Web Vitals speed sweep

Pages load slowly or jump around while loading. Measure, fix, and confirm with Google's speed metrics.

- 1 `/sc:analyze src --focus performance --format report # hot spots`
- 2 `/sc:troubleshoot "LCP/CLS/INP on key pages" --type performance --chrome # measure`
- 3 `/sc:improve src --type performance --loop --iterations 3 # optimise`
- 4 `/sc:test --type e2e --play # still works`

P17 🔒 SECURITY ⚡ POWER

Security hardening before launch

You are about to go live. Find and fix security weaknesses first.

- 1 `/sc:analyze . --focus security -
-depth deep --think-hard #
vulnerabilities`
- 2 `@agent-security-engineer
"threat-model auth, payments,
uploads" # threats`
- 3 `/sc:improve src --type quality -
-focus security --validate #
fixes`
- 4 `/sc:test --type e2e --play #
flows still work`

P18 🗄️ DATA ⚡ POWER

Zero-downtime database change

Change a database that is in use without taking the app offline.

- 1 `/sc:design schema-change --type
database --format spec "expand,
migrate, contract" # plan`
- 2 `/sc:implement "expand migration"
--type service --safe --validate
--with-tests # step 1`
- 3 `/sc:test --type integration #
verify`
- 4 `/sc:implement "backfill +
contract migration" --type
service --safe --validate #
steps 2-3`

P19 📁 BACKEND ⚡ POWER

Contract-first API

Agree the API first, so front-end and back-end can be built at the same time.

- 1 `/sc:design api --type api --
format spec # contract`
- 2 `/sc:spec-panel
@claudedocs/api_spec.md --focus
architecture # review`
- 3 `/sc:implement "API per spec" --
type api --safe --with-tests #
build`
- 4 `/sc:document src/api --type api
docs`

P20 🚀 PERFORMANCE ⚡ POWER

Cut cloud and API costs

Hosting, database or AI bills keep growing. Find where the money goes and reduce it safely.

- 1 `/sc:analyze . --focus
performance --depth deep "API
calls, database queries,
caching, payload sizes, AI model
calls" # where money goes`
- 2 `/sc:research "cost optimisation
for <provider or service>, 2026"
--depth standard # proven
savings`
- 3 `/sc:improve src --type
performance --interactive #
apply savings you approve`
- 4 `/sc:test --type all # nothing
broke`

P21 🔍 QUALITY ⚡ POWER

Review a pull request properly

Before merging someone's changes. Check the code, the risks and whether it really does what the ticket asked.

- 1 `/sc:git diff main # what changed`
- 2 `/sc:analyze src --focus quality
--depth deep "only the files
changed on this branch" # bugs,
style, design`
- 3 `/sc:analyze src --focus security
risky changes`
- 4 `@agent-self-review "check this
branch against the ticket and
acceptance criteria" # does it
do the job?`
- 5 `/sc:test --type all --coverage #
tests still green`

P22 ✅ QUALITY ⚡ POWER

Test suite from zero to solid

The code has few or no tests. Add tests where a bug would hurt most.

- 1 `/sc:test --coverage # baseline`
- 2 `@agent-quality-engineer "risk-
based test strategy for src/" #
strategy`
- 3 `/sc:implement "tests for
highest-risk modules" --type
feature --with-tests # tests`
- 4 `/sc:test --type all --coverage #
after`

P23 🚢 SHIP ⚡ POWER

Release gate

A checklist to run before every release so nothing slips through.

- 1 `/sc:test --type all --coverage #
tests`
- 2 `/sc:analyze . --focus security #
security`
- 3 `/sc:build --type prod --clean --
optimize --validate # build`
- 4 `/sc:git tag v<x.y.z> # tag`

P24 🔄 MIGRATION ⚡ POWER

Modernise or migrate a codebase

Use to move old code to a modern setup, such as JavaScript to TypeScript or a major framework upgrade, in safe steps.

- 1 `/sc:research "<from> to <to>:
breaking changes, codemods,
pitfalls" --depth deep #
migration intel`
- 2 `/sc:spawn "migrate <from> to
<to> in safe phases" --strategy
adaptive --depth deep # phase
plan`
- 3 `/sc:task execute "next phase" --
delegate --task-manage # run a
phase`
- 4 `/sc:improve src --type
maintainability --morph --loop -
-iterations 3 # bulk updates`
- 5 `/sc:test --type all --coverage #
verify`

N1  AI / ML

 NUCLEAR

AI/ML pipeline audit + research-driven optimisation

Use when you want an AI/ML pipeline to be more accurate, faster or cheaper. Ends with a researched, prioritised improvement plan.

- 1 `/sc:index-repo mode=update # map the repo`
- 2 `/sc:analyze ml/ --depth deep --ultrathink --all-mcp --delegate auto "ingestion, features, training, evaluation, serving: accuracy, speed, cost" # full pipeline audit`
- 3 `/sc:research "state of the art + similar open-source pipelines for <task> on <stack>, 2026" --depth exhaustive --strategy unified # latest developments`
- 4 `/sc:design ml-pipeline-v2 --type architecture --format spec --think-hard "apply the findings" # improved design`
- 5 `/sc:workflow claudedocs/ml_pipeline_v2.md --strategy systematic --depth deep # prioritised plan`

💡 Add @agent-python-expert and @agent-performance-engineer for code-level advice.

N2  AUDIT

 NUCLEAR

Audit the entire codebase

Use before a rewrite, handover or big investment. Produces a report on every problem plus a fix plan, and changes nothing.

- 1 `/sc:index-repo mode=update # map`
- 2 `/sc:analyze . --depth deep --ultrathink --all-mcp --delegate auto --format report "architecture, quality, security, performance; every inconsistency" # every issue, by severity`
- 3 `/sc:test --type all --coverage # test health`
- 4 `/sc:spawn "fix plan from the audit reports, worst first" --strategy adaptive --depth deep # fix plan`

💡 Read-only. Nothing changes until you act on the plan.

N3  WEB

 NUCLEAR

Professional website from scratch

Use to build a brand-new website that looks professional, works on every device and is accessible to everyone.

- 1 `/sc:research "2026 UI/UX best practice: accessibility, Core Web Vitals, typography, motion" --depth deep # current standards`
- 2 `/sc:brainstorm "website for <business>: audience, pages, brand, tone" --depth deep # requirements`
- 3 `/sc:design site --type architecture --format spec "pages, design tokens, themes, components" # full design`
- 4 `/sc:implement "complete site from the spec" --type feature --framework react --magic --c7 --with-tests --orchestrate # build it`
- 5 `/sc:improve src --type quality --focus accessibility --loop --iterations 3 --frontend-verify # polish + verify`
- 6 `/sc:build --type prod --clean --optimize --validate # ship-ready build`

💡 Magic needs a 21st.dev API key. Swap react for vue if you prefer.

N4  VERIFY

 NUCLEAR

Prove a feature is complete and wired

Use before calling a feature done, especially after AI-written code. Checks nothing is half-built, unconnected or broken.

- 1 `/sc:reflect --type task --analyze # still on target?`
- 2 `/sc:analyze . --depth deep --ultrathink --all-mcp "TODOs, stubs, mocks, uncalled code, unregistered routes, unread config; trace each feature end to end" # gaps + wiring`
- 3 `@agent-self-review "check every acceptance criterion with evidence" # requirement by requirement`
- 4 `/sc:test --type all --coverage # all tests`
- 5 `/sc:reflect --type completion --validate # final verdict`

💡 Gives evidence for every claim. No tool can honestly promise 100%.

N5 🗑️ CLEANUP

☢️ NUCLEAR

Deep clean everything outdated

Use when the project is cluttered with old code and leftover files. Removes them safely, one approved batch at a time.

- 1 `/sc:git checkout -b chore/deep-cleanup # safe branch`
- 2 `/sc:analyze . --depth deep --ultrathink --delegate auto "deprecated, duplicate, legacy, dead code; stale comments, references, files" # inventory`
- 3 `/sc:cleanup . --type all --interactive "plus caches, builds, temps, logs, artifacts, old audits, unused images; update .gitignore" # remove, batch by batch`
- 4 `/sc:test --type all # still green`
- 5 `/sc:git commit --smart-commit # commit`

💡 Review each list before approving. --aggressive only with a backup.

N6 💬 PROMPTS

☢️ NUCLEAR

Dissect and rebuild all prompts

Use when your app depends on AI prompts. Takes every prompt apart, researches what works best, then rewrites them.

- 1 `/sc:analyze prompts/ --depth deep --ultrathink "role, structure, examples, constraints, output format, cost, failure modes" # full dissection`
- 2 `/sc:research "most effective prompting techniques for <purpose> + official vendor guidance, 2026" --depth exhaustive # best methods`
- 3 `/sc:spec-panel @claudedocs/prompt_audit.md --mode critique # expert critique`
- 4 `/sc:improve prompts/ --type quality --interactive "apply the findings" # rewrite`
- 5 `/sc:test --type all # run your evals`

💡 No eval suite yet? Build one first with P12.

N7 📱 MOBILE

☢️ NUCLEAR

Mobile app from scratch

Use to create a new app for iPhone and Android from one codebase, from first idea to tested screens.

- 1 `/sc:research "React Native vs Flutter for <app>; 2026 mobile UX guidelines" --depth deep # stack + guidelines`
- 2 `/sc:brainstorm "<app idea>: users, screens, offline, notifications" --depth deep # requirements`
- 3 `/sc:design app --type architecture --format spec "screens, navigation, state, API" # design`
- 4 `/sc:implement "app from the spec" --type feature --framework react --c7 -with-tests --orchestrate # build`
- 5 `/sc:test --type all --coverage # verify`

💡 For Flutter, drop --framework and say Flutter in the quotes.

N8 🐛 DEBUG

☢️ NUCLEAR

Catch the bug nobody can catch

Use for bugs that appear randomly and vanish when you look. Builds theories, tests them and fixes the real cause.

- 1 `/sc:troubleshoot "<symptom, frequency, environment>" --type bug --trace -ultrathink --all-mcp --introspect # every hypothesis`
- 2 `@agent-root-cause-analyst "rank the hypotheses, design a test for each" # evidence plan`
- 3 `/sc:troubleshoot "<symptom>" --fix --validate # risk-checked fix`
- 4 `/sc:test --type all # run it many times`

💡 Paste exact errors and logs. Never accept a fix that disables a test.